

HTTP Servlets

Prof. Dr. Peter Braun

9. November 2024

1 Titlepage

HTTP Servlets

Prof. Dr. Peter Braun



No Text – maybe some music :-)

2 Goals of this Unit

Learning Goals

- You know the purpose of a Web application container
- You know how to handle HTTP requests on the server
- You know how to convert Java objects to JSON
- You implement your first non-trivial HTTP Servlet

Welcome to this unit about HTTP servlets. In this unit, we will look at implementing HTTP communication on the server side on a very low level. We assume you learned about the HTTP protocol in the last video. You know what HTTP verbs are and the HTTP request and response message structure. We will program all examples in Java, and in the world of Java, there is a simple tool for implementing the HTTP protocol called Web Application Containers. I will introduce the development of Java Servlets, which are used to implement HTTP-based server applications in Java. We will also talk about JSON and how to use it in Java. Finally, you will develop a small application to manage people's resources.

3 Web Application Container

Web Application Container

- Wait for a network connection on a specific port
- Interpret the incoming message as an HTTP request
- Process the request, for example, read a document
- Create a response message and send it back

Implementing a web server is relatively easy. You set up a TCP/IP connection on the server and practically wait in an endless loop for incoming requests. Then, you interpret these requests by considering the HTTP message format, performing the desired operation, and generating a response in the HTTP response format. Then, you already have everything you need. But things get complicated when you look at the details and must consider all kinds of requests and all possible parameters. Fortunately, there are numerous solutions for this task, some of which are available as open-source and have been tested for many years in production environments. For a web server that can be used mainly for delivering HTML files or images, audio files, and videos, for example, the open-source solution from Apache with the Apache Web Server is suitable. However, we want to go deeper because we want to evaluate incoming requests in detail and generate data dynamically, i.e., load data from a database and deliver it back to the client. In addition, our goal is to understand the HTTP protocol in more detail, and therefore, we use a solution at this point, which also allows us a little more insight. And since we are in the world of Java, we use a so-called Web Application Container. Such a container is ideal for us because it already does much of the work around processing HTTP messages. Such a container can resolve incoming HTTP messages into its components, providing a simple interface for generating response messages. But such a container still leaves us enough freedom to process each request ourselves, i.e., it provides us with certain entry points via Java methods called by the container when specific messages arrive. But we can process the requests ourselves. But why is such a solution named a container? The simple answer is that several such web-based /applications/ should often run on the same computer. The applications are entirely different and have nothing to do with each other. Such a container can now manage several such separate web applications; one can load

each into the container and remove it again without affecting other web applications. We will see how the clients can address the individual applications separately later. There are, again, many different solutions for such a Java-based Web Application Container. A widespread solution from Apache called Tomcat; you might have heard about this. Another example is Jetty from Eclipse, which we are going to use. Such a container is usually started as a standalone application. Installing it on your laptop at home would be laborious for our first experiments. The installation is not complicated, but it takes some time. Therefore, to simplify this unit, we will choose a simpler solution and run a Jetty container embedded in a typical Java application.

4 Run the First Example

Run the First Example

- Clone the repository for this unit
- Import the directory as a new Maven project
- Open class `Main` and run method `main`
- Open `http://localhost:8080/hello` in a browser

Our first example is in a Git repository. Press the button to view the project in a web browser. Then, copy the URL to clone the repository and import the project into your development environment as a Maven project. Open the "Main" class and execute the "main" method. Open a web browser and enter the URL on the slides as the address. You should see some welcome messages in the Web browser.

5 Start the Embedded Jetty Container

Start the Embedded Jetty Container

```
32 public class Main
33 {
34     public static void main( String[] args ) throws Exception
35     {
36         Server server = new Server( 8080 );
37
38         ServletContextHandler context = new
            ServletContextHandler( ServletContextHandler.
                SESSIONS );
39         context.setContextPath( "/" );
40         server.setHandler( context );
41
42         context.addServlet( new ServletHolder( new HelloServlet(
            ) ), "/hello" );
43
44         server.start();
45
46         // The next statement keeps the server running -- don't
            remove it!
47         server.join();
48     }
49 }
```

Now, let's look at the implementation. The class "Main" contains the "main" method that needs to be executed to start up the embedded Jetty container. In line 36, we define which port our Jetty container should wait for incoming requests. Every web application on a Jetty container needs a unique context path defined in lines 39 and 40, which is, in this case, only the "slash" without a specific path name. I mentioned before that, in principle, a container can also manage many different web applications; this context path must then distinguish these. The "Main" class also configures the Java servlets that belong to the Web application. This is done in line 42. The "HelloServlet" waits for incoming requests at path "hello." Therefore, the full path to access this servlet is "slash hello."

6 Processing Incoming Requests

Processing Incoming Requests

- Web container receives an incoming HTTP request
- The matching Web application is selected
- The matching Servlet is selected
- The correct method of the Servlet is selected

What happens when a client sends a request to our embedded Jetty container? After startup, the Jetty waits for incoming requests on port 8080. When a client sends an HTTP request to our server, it is received by the Jetty container, and the message is parsed accordingly. Based on the first path element after the port number and the slash, Jetty identifies the correct web application responsible for handling this request based on the context path. Once again, the context path distinguishes the different Web applications on the server. After that, the Jetty container looks for other path elements to identify the Java class used to process this request in the next step. It then evaluates the HTTP verb in the request and calls an appropriate method within the Java class.

7 The First Java Servlet

The First Java Servlet

```
38 @WebServlet( name = "HelloServlet", urlPatterns = { "docker" } )
39 public class HelloServlet extends HttpServlet
40 {
41     private String message;
42
43     public void init( )
44     {
45         message = "Hello World!";
46     }
47
48     public void doGet( HttpServletRequest request, HttpServletResponse response ) throws IOException
49     {
50         response.setContentType( "text/html" );
51
52         PrintWriter out = response.getWriter();
53         out.println( "<html><body>" );
54         out.println( "<h1>" + message + "</h1>" );
55         out.println( "</body></html>" );
56     }
57
58     public void destroy( )
59     {
60     }
```

To understand this better, let's now look at an implementation of such a Java class. Java classes that can handle HTTP requests are called servlets. Again, as a reminder, our entire web application is accessible under the context path “/” because that is how it was configured in class Main. For each additional path element after “/,” a Java class must now be responsible for processing requests to that path. Classes for processing HTTP requests must inherit from the base class HTTP Servlet. So, in our example, we create a class HelloServlet that extends the class HTTP Servlet. Now, we have to distinguish between two cases. If we call Jetty embedded as shown before, then the path under which this servlet is reachable is defined in class Main. If we call the whole Web application on a standalone Web container, for example, as a Docker container, then the annotation in line 38 becomes important. Through the notation, a class annotation named “WebServlet,” we specify what the servlet is called and, secondly, the most important thing in this line, the name of the path that this servlet should respond to. We will explain the embedded version below. When a request is sent to “/hello,” using the technique we just described, Jetty has identified this class, HelloServlet, as responsible for processing the request. Our first request is a GET request, and accordingly, Jetty looks within our class for the method named “doGet.” You see this in line 48. There might be corresponding methods for the other HTTP verbs. All these methods always get two parameters. The first parameter, named request, contains all the information about the incoming request, and the second parameter, named response, is used to generate the response message. This first small demonstration will not look further at the incoming request. In line 52, we get the object to write in the content area of the response message; in the following lines, we write the text “Hello World” with some simple HTML tags. As a result, the specified text will appear at the client.

8 Exercise 1: Query Parameters in URLs

Exercise 1: Query Parameters in URLs

- Modify the code to accept a query parameter
- The query parameter should be named `name`
- Create a greeting message that contains `name`

Now comes the first programming exercise for you. The task is to evaluate a query parameter given in the URL when processing a GET request and make the response take this parameter into account. So you would have to find out with which method from the object of type `HttpServletRequest` such a query parameter can be read. As a reminder, a query parameter is separated from the path in the URL by a question mark and always consists of a name and a value. An equal sign separates name and value. Of course, you don't have to search the URL itself for such query parameters. The advantage of a Web Container is that it does this work for you. So, you should find the method to read the value from the query by specifying the name. Please call the parameter "name" and expect a person's first name. The response you generate within the processing of this request should then no longer be "Hello World" but "Hello" followed by the passed first name. If you want to see the solution, click "Continue".

9 Screencast 1

Screencast 1

Let's start with the source code of this exercise. We implement a new class here for this exercise, `demoServlet` using query parameters. It has to extend the `HTTP servlet`. And then we only implement the `doGet` method. So far it was already known from the last example how this would work. The open point was for you to find the correct method to get the value of some query parameters here. And of course the value of the query parameters are parsed as part of the `HTTP servlet request object` that is here. And the method that you need to use is `getParameter` here. And you parse the name of the parameter. So of course you don't have to take care of parsing this URL string after the question mark and separate things based on the separator ampersand and the equal sign and so on. And that's pretty much it here. So we have the first name in a local variable. Then we produce the output and we use this variable first name here. So let's start the code here. So this is the main class. And let's remember that we are working in this greeting path here. So we go to postman and we start a get request. Greeting and now we have to parse a parameter. Name is James. And you can see the result here.

10 Exercise 2: Processing a POST Request

Exercise 2: Processing a POST Request

- Implement a Servlet that accepts a POST request
- Read the request body as `String` and save it as `name`
- Create a greeting message that contains the `name`

In the last task, we overrode the method `doGet` to process a GET request. Let's deal with the second HTTP verb in the next task. A POST request is used when data is transferred from the client to the server. Therefore, there is now a method `doPost` that you need to override. In this method, the request parameter contains the data that the client will transfer to the server as part of the request. First, please find out the name of the method that can be used to get the data from the request. Then, do the same as in the last task. You create a response to the client with the text "Hello" followed by the text in the request, so, for example, again, the first name of a person. If you want to view the solution, click "Continue".

11 Screencast 2

Screencast 2

For a POST request we implement a new servlet, `demoServlet` using POST here. It also extends the `HTTP` servlet. And for a POST request we have to override this `DOPOST` method. The parameters are the same. And what we do here is we use the incoming request. We call method `getReader` to read from the body of the request. And we make use of a helper class here, `ioutils`. This comes from an Apache package. And this class provides a lot of methods that make it easier for us to transfer streams, like this buffered reader here, into a string. And without any parameter name or key or so, we just read the full content of the body. And we store this in this variable `firstName`. And the rest is as usual, we use this `firstName` to create the response method. And we can start this. Because I want to demonstrate how you can call this servlet using postman. The URL is the same, but we have now to send a POST request. And when you send a POST request you have to use this tab here, body, where you have some options how the body should be configured. And for this example we go for a raw content and we just pass the content here. And we send the request. And that's the result.

12 JSON Representation

JSON Representation

```
1 {  
2   "firstname": "James",  
3   "lastname": "Bond",  
4   "city": "London"  
5 }
```

- JSON = JavaScript Object Notation
- It is usually (much) smaller than XML
- It can be used in JavaScript by `JSON.parse()`
- We will use the Java library Jackson

Next, let's look at how to exchange not just a single data, for example, the first name, between client and server, but more complex data structures. You've probably heard of XML or JSON as data formats that we can use to structure many data elements. Let's first repeat some basic things about JSON. You must have heard that the abbreviation JSON stands for JavaScript Object Notation. It has the great advantage that in clients programmed with JavaScript, such data can be used immediately as JavaScript objects. In JSON, individual data sets are understood as objects delimited with curly brackets. The attribute names, also named keys, are separated from the values by colons. The attributes or keys must always be written in quotation marks; this is only necessary for the values if they are strings. Commas separate the individual key-value pairs.

13 Collection Representations

Collection Representations

```
1 [
2   {
3     "firstname": "James",
4     "lastname": "Bond",
5     "city": "London"
6   },
7   {
8     "firstname": "Ernst Stavro",
9     "lastname": "Blofeld",
10    "city": "Piz Gloria"
11  }
12 ]
```

- A collection of objects is written with square brackets

If you want to combine several objects into an enumeration, use a notation that strongly reminds you of arrays. The enumeration starts with a square bracket and ends with a square bracket. The individual objects are delimited as usual with braces and separated by commas.

14 Serialization – the Simple Way

Serialization – the Simple Way

■ We use the ObjectMapper from Jackson

```
1 ObjectMapper mapper = new ObjectMapper();
```

■ A Java object of type Map can be serialized into JSON

```
26 private static void serializeMaps() throws Exception
27 {
28     final var james = new HashMap<>();
29     james.put( "firstName", "James" );
30     james.put( "lastName", "Bond" );
31
32     final var result = mapper.writeValueAsString( james );
33     System.out.println( result );
34 }
```

Numerous libraries are available for data serialization and deserialization. Serialization means transferring data from the internal format as Java objects to JSON. These libraries do more or less the same, and we go for the Jackson Library in this unit. It depends very much on your taste, preferences, or project requirements and which library you want to use. In our first example, we start very simple and want to convert a simple map into JSON format. First, you have to create an instance of type `ObjectMapper`. This class is the main class responsible for data serialization and deserialization. We make a map in lines 28-30; in line 32, we call the method `"writeValueAsString"`, pass the map as a parameter and get a valid JSON String. To check, let's display the result on the screen.

15 Deserialization – the Simple Way

Deserialization – the Simple Way

```
38 private static void deserializeMaps() throws Exception
39 {
40     final var jsonOfPerson = "{\"firstName\":\"James\",\"lastName\":\"Bond\"}";
41     final var map = mapper.readValue( jsonOfPerson, Map.class );
42
43     System.out.println( map.containsKey( "firstName" ) );
44     System.out.println( map.containsKey( "lastName" ) );
45 }
```

- JSON can be deserialized into Java Maps
- In the case of embedded objects, the value contains another Map

The opposite direction is called “deserialization” and can be implemented quickly. We manually create an object in line 40 in the JSON formatting for demonstration. Don’t be irritated by the many backslashes; they only need to process the quotes correctly. In line 5, we call the method “readValue”. Here, we have to give the JSON formatted string as a parameter and a hint of what kind of Java object we want to get. In this first example, we want to get back a map. This looks good on the first try, but this straightforward approach has a disadvantage. When unpacking JSON and storing the data in a map, we unfortunately lose any information about the values of the attributes. The keys of the map are always strings, but for the values, we can’t assume anything more than that they will be objects. So, if we access values in the map using the name as a key, we may have to transform it into the correct data type ourselves.

16 Example of a Simple Java Class

Example of a Simple Java Class

```
1 public class Person {  
2     private String firstName;  
3     private String lastName;  
4  
5     public Person() {  
6         super();  
7     }  
8  
9     public Person(String firstName, String lastName) {  
10        this();  
11        this.firstName = firstName;  
12        this.lastName = lastName;  
13    }  
14  
15    // Getter and Setter methods  
16 }
```

- Remember that you always need an empty constructor!

But of course, there is a more elegant way. Let's now look at converting any Java object into the JSON format. For example, let's create a class called Person, which has two object attributes for the first and last names. Every class to be transferred automatically with Jackson in the following must have an empty constructor. You can see the example in lines 5-7. For clarity, we have omitted the getter and setter methods in this example, but they are necessary.

17 Serialization of Java Objects

Serialization of Java Objects

```
49 private static void serializeObjects() throws Exception
50 {
51     Person person = new Person( "James", "Bond" );
52
53     String jsonOfPerson = mapper.writeValueAsString( person );
54     System.out.println( jsonOfPerson );
55 }
```

We can now transfer an object of type `Person` into the JSON format. We create such an object of type `Person` in line 51. Then, we can call the “`writeValueAsString`” method in line 53 and give this person as a parameter. This is extremely easy for us as developers; the library does a lot of work for us here. You should understand the functionality used here to convert any Jackson object to JSON for the whole thing to work.

18 Rules for Serialization and Deserialization

Rules for Serialization and Deserialization

- Jackson can access `public` or `package` attributes
 - It also uses public `Getter` and `Setter` methods
 - No `private` attributes are processed
 - The Java class must have an empty constructor
-
- Use `@JsonIgnore` to exclude attributes
 - Use `@JsonProperty` to rename attributes

All attributes with visibility `Public` or `Package` can be processed by Jackson. However, the use of public `getter` and `setter` methods is preferred. Unfortunately, private methods or private attributes cannot be processed. Every class to be processed this way must contain an empty constructor, i.e., a constructor without parameters, because only with this is it possible later to create a Java object again from a JSON string. Some notations can still control the translation of the Java objects into the JSON format. For example, with the annotation `@JsonIgnore`, you can exclude individual attributes or the `getter` and `setter` methods from the conversion, or with the annotation `@JsonProperty`, you can rename the attributes. This is important, for example, if you want to name an attribute differently in your Java implementation, but your communication partner still expects the old name.

19 Deserialization of Java Objects

Deserialization of Java Objects

```
59 private static void deserializeObjects() throws Exception
60 {
61     final var jsonOfPerson = "{\"firstName\":\"James\",\"lastName\":\"Bond\"}";
62     final var person = mapper.readValue( jsonOfPerson, Person.class );
63
64     System.out.println( person );
65 }
```

- But, you can see, you always need a corresponding Java class!

Finally, I would like to show you how to create a Java object from a JSON string. In line 61, we create such a string again as a test and then call the method to deserialize in line 62. We now have to specify the concrete class for which we want to create an object as the second parameter. When unpacking, the library proceeds as follows: for each attribute in JSON format, a corresponding method is searched for in the specified class and then called dynamically at runtime.

20 Deserialization Collections of Objects

Deserialize Collections of Objects

```
79 private static void deserializeCollections() throws Exception
80 {
81     final var jsonOfPerson =
82         "[{"firstName\":\"James\", \"lastName\":\"Bond\"}, {"firstName
83             \"Ernst\", \"lastName\":\"Blofeld\"}]";
84     // @formatter:off
85     final var persons = mapper.readValue( jsonOfPerson, new
86         TypeReference<List<Person>>() {} );
87     // @formatter:on
88     System.out.println( persons );
89 }
```

Serialization of Java collections such as lists or sets is easy with Jackson. However, deserialization is more complex to implement. Therefore, here are some source code examples for you beside me. The most important line is 84. The problem is that we, as developers, must give Jackson the information on the result type. Because of some limitations of the Java language, the information about generic types in collections is missing at runtime if we don't use this trick shown in line 84. We have to wrap the destination type "List of Person" as a generic type into a, let's say, a container with the name "TypeReference." At this point, we not only give type information, you can see there is no ".class" at the end, but we create an object that keeps all the information about the generic types at runtime.

21 Exercise 3: A Simple Service

Exercise 3: A Simple Service

- Implement class to manage a collection of persons
- Clients can create a new `Person` object using POST
- Clients can request a specific `Person` using GET
- Clients can request all `Person` objects using GET
- Clients can filter `Person` by first and last name

Next, we come to the last exercise about a very simple management of persons. Based on what you learned today about implementing Java servlets and data handling with Jackson, you should now be able to implement a Java servlet that implements the following requirements. The servlet accepts POST requests to create new people. The servlet accepts incoming GET requests for single-person objects and all objects. In addition, it should also be possible to filter by first and last name. Go to the corresponding branch for this exercise; some classes have already been implemented. Click "Continue" if you want to see the solution.

22 Screencast 3

Screencast 3

For this first and still quite simple service we implement a servlet, a new servlet that extends HTTP servlet of course. Since we work with JSON representations in this example here, in this exercise, we initialize an object attribute, object mapper, that's the Jackson library for JSON serialization and deserialization and we initialize this here. In the constructor of this servlet we register a module that takes care of formatting dates and timestamps. This servlet only has two HTTP verbs implemented, get and post and for the get case we have to distinguish between the get single and the get collection case. This is implemented here, we have the do get method, this is the decision for get single request, and we can have a look at the implementation here, is path ID set, uses method get path info from an HTTP servlet request class and this method get path info looks at the path that follows the path element that points to the servlet and before the question mark if there is any that would indicate that there are some query parameters now. So this is the path element where the ID should be present if it's a get single request and if there is no such path element then this method returns null and this is our indicator that it's a get collection request. If it's a get single request we go to this method here, if it's a get collection request we consider it as a query and there might be some query parameters but not necessarily and we go to the do get by ID method first so here in this line we read the ID then we ask the storage to return the person with this ID and if there is a person with this ID then we convert this to a JSON and otherwise we return 404 which is the status code for not found. If it's a query we request the two query parameters and we pass this to a method called find by of the storage and we convert this to JSON. This conversion to JSON is implemented here. The second parameter is of type object because we don't distinguish the two cases that it's a single response or collection response. We set the content type of the response header to application JSON which is

the correct media type to indicate that we are now transmitting JSON and we use the object mapper to convert the result into a string that is formatted according to the JSON rules. So that's everything for the two get cases and the post case is implemented here. In this line we read the body of the request, we assume that it is a JSON formatted string, we use the object mapper to convert this to a new person object, we create a new person in the storage and now we must create a URL that points to this new person and here you must be careful because of these two cases that we have a docker system running or a standalone system running. So the correct URL would be like this for the case that you start up the servlet in this embedded JT container. Very important here is the correct status code, so 201 indicates that a new object was created, the name of this status code is created and then we have to pass the URL that we created here as parameter under the name location in the response header. So let's start the server and we start with the first get request and you can see there is already something in the database or in the storage and let's check why this is, why it is. So we go to the implementation and we have this person storage class here, this is simply a map of key to person object and when we create this person storage object we populate the internal map here and populate means we create something that we can work with existing data and here we create exactly one object. Otherwise this class has the basic CRUD operations like create, read, update, delete and also a find by method that is for filtering. So that's the get collection request here, we can now ask for a single object and now I would like to point you to one kind of problem in this implementation because if we write this here, so persons as the path element that points to the servlet and then a slash then this happens 404 not found and this is because there is some minor let's say error in the implementation which is I can show you this here which is in this is path ID set method because in this case that we have a slash the path info is not null. So path info is not null that means it is an ID request but as you can see the ID is missing and when the ID is missing then we convert everything after the slash and if there is nothing this method returns an object of type optional which is sorry which is empty and since we don't assume let's check what we assume here as default value, no it's an optional so when we call this method here, yeah we check that if the ID is present then we go to the storage and ask for the ID otherwise the option is empty and if the option is empty we return the 404 here. So we don't improve the implementation now because this is the way we implement the get the do get method only in this very exercise from the next unit on we use another framework which takes care of these kind of simple cases. So I showed you the get single request, the get collection request and the only thing that is missing is the post request and for the post request I will copy this data here and now we go to the post request we have to set a raw body I copy this and I just change some data and very important leave out the ID we get a 201 created and let's check here the location header is set with a correct URL so the ID of this new object is 2 and we send a get single request to this and this is the result here and we can also ask for all personal objects and then you can see we have a collection here of two objects.

23 Summary

Summary

- Java Servlets to handle HTTP request
- Jackson to serialize and deserialize Java objects
- Use cases for GET and POST requests
- Your first non-trivial HTTP service

That's it for this unit. Today, you learned how to implement a simple service using Java Servlets. In doing so, we first looked only at GET and POST requests, and you saw how you can use the Jackson library to transfer data structures into JSON format very easily. However, you have also noticed that some manual work is required to implement more complex services in this way. In a later unit, you will learn about other technologies that make this easier to implement.

24 Literature

Literature

- Rainer Oechsle: Parallele und verteilte Anwendungen in Java. Hanser, 5. neu bearbeitete Auflage, 2018.
- William Crawford, Jason Hunter: Java Servlet Programming – Help for Server Side Java Developers (English Edition), 2nd edition, O'Reilly, 2001.